TPRF: A Transformer-based Pseudo-Relevance Feedback Model for Efficient and Effective Retrieval

Hang Li*
hang.li@uq.edu.au
University of Queensland
Brisbane, Australia

Chuting Yu*
chutingyu.cs@gmail.com
University of Queensland
Brisbane, Australia

Ahmed Mourad
a.mourad@uq.edu.au
University of Queensland
Brisbane, Australia

Bevan Koopman
bevan.koopman@csiro.au
CSIRO
Brisbane, Australia

Guido Zuccon
g.zuccon@uq.edu.au
University of Queensland
Brisbane, Australia

ABSTRACT

This paper considers Pseudo-Relevance Feedback (PRF) methods for dense retrievers in a resource constrained environment such as that of cheap cloud instances or embedded systems (e.g., smartphones and smartwatches), where memory and CPU are limited and GPUs are not present. For this, we propose a transformer-based PRF method (TPRF), which has a much smaller memory footprint and faster inference time compared to other deep language models that employ PRF mechanisms, with a marginal effectiveness loss. TPRF learns how to effectively combine the relevance feedback signals from dense passage representations. Specifically, TPRF provides a mechanism for modelling relationships and weights between the query and the relevance feedback signals. The method is agnostic to the specific dense representation used and thus can be generally applied to any dense retriever.

ACM Reference Format:
Hang Li*, Chuting Yu*, Ahmed Mourad, Bevan Koopman, and Guido Zuccon. 2022. TPRF: A Transformer-based Pseudo-Relevance Feedback Model for Efficient and Effective Retrieval. In *Proceedings of the 26th XXXX, December 15–16, 2022, Minneapolis, MN, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/xxxxxx

1 INTRODUCTION

Pseudo-relevance feedback (PRF) aims to reduce the effect of query-passage vocabulary mismatch and thus improve search effectiveness by modifying the original query using information from top-ranked passages. PRF has been widely investigated in the context of traditional bag-of-words retrieval models [1].

Transformer [21] based pre-trained deep language models [4, 5, 17, 18, 26] have shown promising results and improvements in information retrieval related tasks, and in particular for the cross-encoder architecture [6, 27]. PRF methods for this architecture have also been proposed [15, 22, 28, 31, 32]. However, cross-encoder methods are computationally expensive, requiring both query and passage encoding at query time. This limitation also applies to PRF methods for this architecture.

To overcome the computational cost limitation of cross-encoders, an alternative transformer based architecture has been devised: the bi-encoders, used by dense retrievers [7, 8, 12, 13, 19, 25, 30]. In such an architecture, the passage encoder and the query encoder are trained separately, passages are encoded at indexing time, and thus at inference time (i.e. querying time) only the query requires encoding. Empirically, dense retrievers achieve similar or even better effectiveness compared tp cross-encoder methods but with much higher efficiency [16, 20].

The use of the PRF process in the context of dense retrievers has seen some initial work [11, 23, 29], but it is still a problem that is largely unexplored. Generally speaking, these methods consider the signal from the PRF passages in combination with the original query, input into a BERT-based query encoder fine-tuned for the PRF setting, and produce a new query encoding which is then matched against passages via dot product to generate a final ranking. For example, a current state-of-the-art PRF techniques for dense retrievers [11, 29], known as ANCE-PRF [29] in its setting that builds on top of the ANCE dense retriever [25], concatenates the text of query and feedback passages and provides them as input to an ANCE-based query encoder that has been fine-tuned for the PRF setting. Other dense retriever PRF methods differ in, among other aspects, the dense encoder used (e.g., ColBERT-PRF [23]) and the use of dense vectors as input to the dense retriever encoder in place of text (Vector-based PRF [10]). Common drawbacks of these PRF methods are that: (1) the encoder used for the PRF signal is based on large pre-trained language models like BERT (for ColBERT-PRF, RepBERT-PRF [11]) and RoBERTa (for ANCE-PRF), rendering this encoder quite large in size (e.g., 503.4 MB for ANCE-PRF); and (2) the PRF signal given as input to the PRF encoder is often represented as text and thus requires expensive BERT inference operations to perform the PRF process, especially if CPUs, rather than GPUs, are used. Recall that the computational cost of a BERT inference grows as the size of the input grows. Thus the encoding of the query text plus the text of several passages is far more time consuming than the encoding the query alone.

In this paper, we consider the problem of devising PRF techniques for dense retrievers for systems with limited memory and

*Both authors contributed equally to this work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
XXXX '22, December 15–16, 2022, Minneapolis, MN, USA
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 999-9-9999-9999-9/99/99…$15.00
https://doi.org/xxxxxx

computational resources, like cheap cloud instances or smartphones and smartwatches, while improving the effectiveness of the dense retriever to which PRF is applied and without adding considerable query latency. These small cloud instances and embedded systems are typically not equipped with GPUs and their memory is limited (e.g., 8GB for an Apple Watch and 1GB for a $0.0104/hour AWS T3.micro instance). In these settings, the use of current PRF methods for dense retrievers is often hampered by the model size or the high query latency on CPUs, and one may be prepared to forgo some of the effectiveness that these state-of-the-art PRF methods can deliver for smaller and quicker, and thus cheaper, models (but that still improve on the baseline dense retriever).

To tackle the challenges presented in this resource-constrained environment, we propose a transformer-based PRF method, called TPRF, which learns how to effectively combine the relevance feedback signal from dense passage representations without relying on PRF encoders based on large pre-trained language models. Specifically, TPRF provides a mechanism for modelling the relationships and the weights between the query and the relevance feedback signals. To address the efficiency issue, in place of a large pre-trained transformer-based language model PRF encoder, we use a much smaller transformer with fewer layers and attention heads. With our specifically designed training regime, the loss in effectiveness compared to larger and more costly PRF methods for dense retrievers is often marginal. This research directly addresses the drawbacks discussed above.

2 METHODS

Next we describe our Transformer-based PRF (TPRF) method. TPRF addresses the limitations of current PRF models for dense retrievers in terms of model size and efficiency by replacing the use of pre-trained deep language models as feedback signal encoders with purposely built transformer models. This is achieved by considering dense representations, rather than text, as the relevance feedback input signal.

2.1 Transformer-based PRF

The TPRF method consists of a dense retriever, a transformer, and a purposely devised training strategy for the transformer.

The dense retriever is responsible for producing a dense vector representation for each passage in the corpus. These are produced at indexing time and stored in a data structure that supports fast Approximate Nearest Neighbor (ANN) search, like a Faiss index. The dense retriever also produces a dense encoding of the initial query and a first ranking of the passages in answer to the query. The TPRF method does not prescribe which dense retriever need to be used – and it is in fact agnostic to the dense retriever used. In the experiments in this paper we demonstrate TPRF in conjunction with the dense retrievers ANCE [25] (and we indicate this method as ANCE-TPRF) and directly compare it against the ANCE-PRF method [29].

Once the first round of retrieval has been performed, the dense representations (and not the text) of the top-k passages are used as feedback signal (i.e., PRF depth = k). The dense representation of the query is then stacked with the representations of the feedback signal to form a $(k+1, 768)$ matrix. This matrix is provided as input to the transformer at the core of the TPRF model. The output is a $1,768$ dimensions dense vector representation. This output is used as the new query representation from the PRF mechanism and used to perform a second round of retrieval using the original dense vector index: its result is the final ranking produced by TPRF.

At the core of TPRF is the transformer that is used to convert the input query representation and feedback passages representations into a new representation of the query with feedback. We choose the vanilla transformer encoder layer proposed by Vaswani et al. [21] to build the PRF query encoder in TPRF. Similar to other transformer layers used in language modelling tasks, the transformer encoder used in TPRF also leverages the self-attention mechanism to transform a sequence of input embeddings into contextualized embeddings. In our context, the transformer converts the query representation and the feedback passages representations into a new query representation. We inherit the key components of the vanilla transformer encoder layer, namely the positional encoding and the multi-head attention layer, to form the transformer encoder layers used in the TPRF query encoder. We apply the same positional encoding of Vaswani et al. [21] on the input feedback passages embeddings to encode the rank position of the feedback passages returned from the first round of retrieval. The multi-head attention layer then produces the input query embedding and the feedback passage embeddings interact with each other and aggregate information via self-attention. As per common practice, the output of the multi-head attention layer follows a feed forward layer with residual connection and layer normalization. Finally, we stack multiple transformer encoder layers to build the new query encoder. In our experiments, we study how different numbers of heads in the multi-head attention layers and different numbers of transformer encoder layers affect the effectiveness of TPRF.

2.2 Training TPRF with Hard Negative Sampling

At training time, after the initial retrieval using the baseline dense retriever, we pick the top-k passages as the PRF, and we randomly sample 20 negatives among the passages that rank between position 10 and 200 in the initial ranking produced by the dense retriever. This sampling strategy was also used by Yu et al. [29] for ANCE-PRF. We sample one target passage vector from the labelled relevant passages from the training dataset (MS MARCO training set queries, see Section 3).

For each training example, we randomly sample one positive passage p^+ and a set of negative passages $\{p_1^-, p_2^-, ..., p_{m-1}^-\}$ from the top passages retrieved by dense retriever; these labelled passages train the TPRF transformer with a supervised cross-entropy loss (Eq. 1); the similarities between query, positive (target) passage, and negative passages are calculated using dot-product:

$$\mathcal{L}_{CE} = -\log \frac{e^{s(q,p^+)}}{e^{s(q,p^+)} + \sum_{p^-} e^{s(q,p^-)}} \tag{1}$$

where $s(q,p)$ is the dot product of a query embedding q and a passage embedding p.

2.3 Theoretical Advantages of TPRF

We highlight that our TPRF uses a vanilla transformer model, i.e. the model is not pre-trained on a different task and data before

being trained for PRF. This is unlike methods such as ANCE-PRF and its variants [11, 29], that rely on pre-trained language models (RoBERTa) and the already fine-tuned ANCE encoder. This means the transformer in the TPRF model can be kept quite compact in size (number of parameters), thus resulting in a model that can potentially require less memory and less encoding time. Another advantage of TPRF is that its input are dense vectors, not text: this means that TPRF is not limited in the size of the input signal, i.e. the number of feedback passages. This is unlike ANCE-PRF and its variants, for which the feedback signal consists of the text of the top passages and thus is bounded by the maximum input size of the pre-trained language model.

3 EXPERIMENTAL SETUP

3.1 Datasets and Evaluation Metrics

We experiment with TREC Deep Learning 2019 [2], and 2020 [3] (TREC DL 2019, TREC DL 2020). These datasets share the same passage corpus (MS MARCO [14]), which consists of ≈ 8.8M passages crawled by the Bing search engine, but with different queries (43 queries for 2019 and 54 for 2020) and graded relevance assessments. We do not use the MS MARCO queries for evaluation. These queries are characterised by shallow judgements: on average only one labelled (and relevant) passage per query; no non-relevant labels are provided and thus is unsuitable to evaluate recall, which is often the target of improvement for PRF methods. We instead use queries in the MS MARCO train set for training the considered methods.

As for evaluation metrics, we use Average Precision (MAP), Reciprocal Rank (RR), Recall@1000 (R@1000), and normalised Discounted Cumulative Gain at various rank cut-off (nDCG@1,3,10,100). For all evaluation metrics, statistical significance testing using a paired two-tailed t-test with Bonferroni correction is performed.

Along with effectiveness, we also measure the efficiency in terms of query latency of all methods in a CPU environment consisting of a consumer-grade Apple computer witg 2.4GHz 8-Core Intel Core i9 CPU and 64GB 2667MHz DDR4 memory. For measuring query latency, we randomly sampled 100 queries from the MS MARCO dev set, then issue them to each method, and report the average query latency measured. Finally, we also record the dimensions of each models in MB.

3.2 Models

For comparison, we report the effectiveness of the ANCE dense retriever [25]; we also use ANCE to implement the ANCE-PRF method and we use ANCE also as the first stage model and source of dense representations for our ANCE-TPRF. For ANCE and ANCE-PRF we use the checkpoints provided by the corresponding authors; we also use Faiss [9] to build the dense index.

3.3 TPRF Implementation and Training

We implemented the vanilla transformer model in TPRF using Pytorch, Pytorch Lightning, and the Huggingface transformers library [24] and separately combine TPRF with ANCE.During the training process, we freeze the original query and passage encoders of ANCE: the query and passage representations generated from ANCE are not changed. We investigate different values of transformer layers (6, 8, 10, 12) and attention heads (4, 6, 12). We additionally also train a transformer with just one layer and one head, as the smallest possible instance of TPRF. The hidden layer is fixed to 1024, input dimension to 768, the dropout to 0.2, and the learning rate to 1e-5. We set the batch size to 512, and trained for 50 epochs with AdamW optimizer, selecting the model with the highest nDCG@10 (following ANCE-PRF) on MS MARCO dev set, which was used for validation. All models were trained on a single NVIDIA Tesla SMX2 32GB GPU; training each model took approximately 26 hours for 50 epochs. For comparison, we execute all PRF methods (ANCE-PRF and ANCE-TPRF) with PRF depth $k = 3$: this is the best configuration for ANCE-PRF and the checkpoint distributed for this model refers to $k = 3$. We highlight that the effectiveness of ANCE-TPRF may also vary largely depending on k; we leave this investigation to future work.

4 RESULTS

4.1 Overall Effectiveness

Table 1 reports the effectiveness of ANCE, ANCE-PRF and ANCE-TPRF on TREC DL 2019 and 2020. For ANCE-TPRF we report the model configuration that obtained the highest nDCG@10 on the validation set: this is the model with 6 transformer layers and 12 attention heads.

From the results, we observe that ANCE-TPRF always outperforms ANCE except in nDCG@3 for DL 2019 (only 0.3% loss) and in MRR and nDCG@k in DL 2020. These differences are however not statistically significant. Differences between ANCE-PRF and ANCE-TPRF are more substantial. In particular, ANCE-TPRF is consistently outperformed by ANCE-PRF and some of the differences are significant. If one was interested in effectiveness alone, then it is clear ANCE-PRF is a better method, and that ANCE-TPRF does provide some gains over ANCE especially for metrics that value some form of recall (R@1000, MAP).

4.2 Trade-off between Effectiveness, Model Size and Query Latency

The motivation for proposing TPRF, however, was to achieve a balance between the improvements in effectiveness promised by PRF and the need for efficiency (in query latency and memory consumption) required by the resource constraint environment we are interested in. Next, then, we study the trade-offs between effectiveness, query latency and model size. This analysis is reported in Figure 1. For TPRF-ANCE, we report the best checkpoint (according to nDCG@10 on validation) for each configuration of number of transformer layers and number of attention heads. From the figure we can make the following observations:

- Concerning the variation in effectiveness across ANCE-TPRF models, we note that their differences are not large for nDCG@10 across both datasets: there is no statistical significant difference between the best and the worst performing model. The differences in terms of Recall@1000 are more marked, especially for DL 2019, where (1) models seem cluster into two groups in terms of effectiveness, (2) differences between the best and the worst model are significant.
- ANCE-PRF consistently outperforms ANCE-TPRF across metrics and datasets, with the exception of Recall@1000 on DL 2019, for which differences are minor. Despite this, differences between the

	TREC DL 2019				TREC DL 2020			
	MAP	R@1000	MRR	nDCG@10	MAP	R@1000	MRR	nDCG@10
[a] ANCE	0.3710	0.7554	0.8372	0.6452	0.4076	0.7764	0.7907	0.6458
[p] ANCE-PRF	**0.4253**$^{+a}$	**0.7912**$^{+a}$	**0.8492**	**0.6807**	**0.4452**$^{+a}$	**0.8148**$^{+a}$	**0.8371**$^{+a}$	**0.6948**$^{+a}$
[t] TPRF-ANCE	0.3855^{-p}	0.7771^{+a}	0.8358	0.6578	0.4135^{-p}	0.7865	0.7743^{-p}	0.6407^{-p}

Table 1: Effectiveness of dense PRF methods; bold values are the best for that metric and dataset. Statistical significant differences ($p < 0.05$) are indicated using the superscript character corresponding to each method.

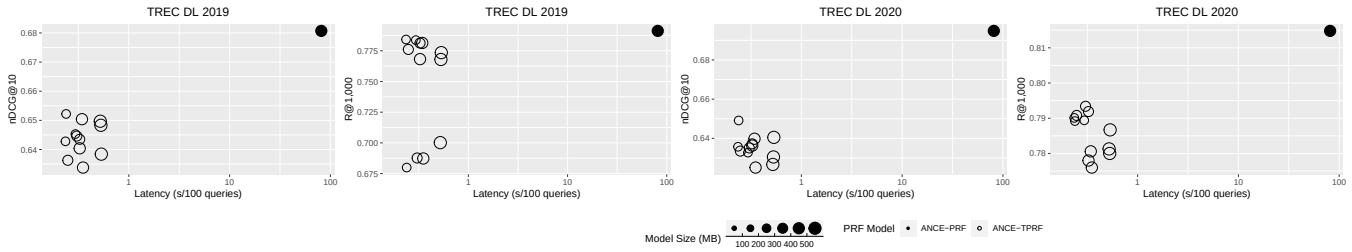


Figure 1: Relationship between effectiveness (measured as nDCG@10 and R@1000), query latency and model size for PRF methods, across datasets. For ANCE-TPRF, we display different configurations (w.r.t. number of layers and attention heads).

best ANCE-TPRF model and ANCE-PRF are not significant for DL 2019; and they are only significant in DL 2020 for nDCG@10.

- However, the larger effectiveness of ANCE-PRF comes at a significant cost: higher query latency. In fact, ANCE-TPRF is blazing fast on a commodity CPU, with 100 queries taking a fraction of a second to run. ANCE-PRF instead requires in the proximity of 100 seconds to run the same amount of queries.
- In terms of model size, it is interesting that for ANCE-TPRF, model size and effectiveness are not correlated. In fact, it is often the case that the best models span different sizes, and that small models perform quite well and at times better than larger models – for example, the ANCE-TPRF model with highest nDCG@10 on DL 2020 (0.6491) has a size of 299.2 MB, while the largest model has size 582.9 MB and achieves a nDCG@10 of 0.5581.
- The ANCE-TPRF models that are closest in effectiveness to ANCE-PRF exhibit less memory consumption than ANCE-PRF. The size of ANCE-PRF is 503.4 MB. The best ANCE-TPRF model for DL 2019 on both nDCG@10 and R@1000 is 299.2 MB; the best ANCE-TPRF model for DL 2020 is 299.2 MB for nDCG@10 and 393.8 MB for R@1000.
- For TPRF, model size is determined by the number of transformer layers l, rather than that of attention heads h. For example, the smallest models are 299.2 MB in size and are obtained with the combinations $(l = 6, h = 4)$, $(l = 6, h = 6)$ and $(l = 6, h = 12)$.
- The TPRF models with less memory footprint are also the ones with the lowest query latency. Although the query latency of the smallest models (0.235 s/100 queries, 299.2 MB) is almost half that of the the largest (0.533 s/100 queries, 582.9 MB), the absolute difference is modest (≈ 0.30), compared to the query latency of ANCE-PRF.

The model with 1 transformer layer and 1 attention head, not included in the analysis above, deserves particular consideration. This ANCE-TPRF setting is characterised by the smallest transformer, at just 62.7 MB in size, and the smallest query latency, at 0.185 s/100 queries. Although not as complex as ANCE-PRF or the other ANCE-TPRF models, this model is still able to improve ANCE in terms of MAP and R@1000 on both datasets.

In summary, ANCE-PRF obtains the highest effectiveness across metrics and datasets, while ANCE-TPRF trades-off higher effectiveness for smaller and faster models. This trade-off is especially useful in memory-constrained settings: one can train a smaller TPRF model, thus forgoing some effectiveness, but still can get improvements over the base dense retriever models.

4.3 Query Latency and Scalability to PRF Depth

In ANCE-PRF, query latency is determined by the length of the input (query text plus text of the appended top k passages used as feedback): when the input length increases, the inference time also drastically increases. This is because inference speed is impacted bu the self-attention computation: this is an expensive process [4, 21] (in terms of runtime) and larger the length of the input, more self-attention computations are required. The input of ANCE-PRF however is also bounded by the maximum size of the input to RoBERTA ($\approx$ 512 tokens): ANCE-PRF's latency will never surpass a certain limit: in our experiments, this occurs for $k = 5$ for which latency is $\approx$ 125 s/100 queries. This happens because any part of the input that surpasses the input size limit is truncated and thus ignored.

In ANCE-TPRF, query latency increases with the increase of k, i.e. the number of feedback passages. However, the TPRF inference time is substantially lower than that of ANCE-PRF, regardless of k.

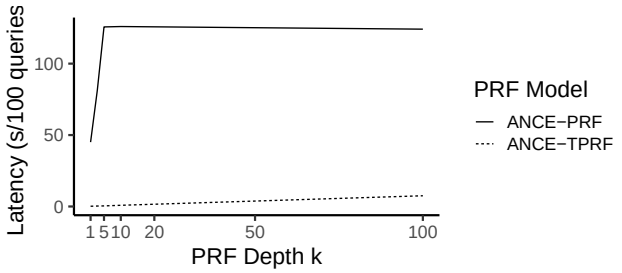


Figure 2: Query latency for the PRF models as a function of the PRF depth k.

This is because, while in ANCE-PRF larger k values mean more text to encode (so more time required), in ANCE-TPRF larger k values simply mean more dense vectors been accumulated in the input – and increases in this number do not determine increases of the transformer's inference time, which stays the same. Instead, as k increases, the time taken to form the input matrix increases, but this is negligible. We demonstrate these remarks in Figure 2, where the PRF depth k is varied up to 100. ANCE-PRF query latency increases very rapidly as k increases, but then plateaus when $k = 5$ because the input size saturates (i.e. exceeds the maximum of 512 tokens). The query latency of ANCE-TPRF also increases as k increases, but this increase is very slow-paced; in addition ANCE-TPRF is not limiteed in input size. Even after $k = 100$ passages have been given as input to ANCE-TPRF, its query latency is still largely less than the query latency of ANCE-PRF when $k = 5$.

5 CONCLUSION

In this paper, we consider pseudo relevance feedback methods for dense retrievers in a resource constrained environment. Specifically, we observe that current PRF methods based on large pre-trained language models and fine-tuned dense retrievers query encoders are not suitable to resource constrained environments such as embedded systems with limited memory and no GPU. These models in fact are often very large, and their query latency on CPU can be considerable – especially if the feedback signal is large or the CPU is not powerful.

To overcome these challenge we devise a transformer-based PRF approach, called TPRF, which leverages transformer encoder layers that do not use the feedback passage text as input, but their dense representations. This allows us to produce more compact models (i.e. smaller in size), and because they have less parameters they also guarantee a lower query latency, thus being usable in settings with low memory and low CPU capabilities. In addition, our TPRF method addresses some of the limitations of previous PRF models, such as the limited amount of PRF signal they can handle – what is more, TPRF is highly scalable to increasing input signal. Empirical results in the context of the ANCE dense retriever confirm that our TPRF method is characterised by lower query latency and memory utilisation than other PRF methods, while having to forgo often only minimal effectiveness (i.e. no statistical significant losses).

REFERENCES

[1] Hiteshwar Kumar Azad and Akshay Deepak. 2019. Query expansion techniques for information retrieval: a survey. *Information Processing & Management* 56, 5 (2019), 1698–1735.
[2] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M Voorhees. 2020. Overview of the TREC 2019 Deep Learning Track. In *Text REtrieval Conference, TREC*.
[3] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M Voorhees. 2021. Overview of the TREC 2020 Deep Learning Track. In *Text REtrieval Conference, TREC*.
[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 2978–2988.
[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.
[6] Jiafeng Guo, Yixing Fan, Liang Pang, Liu Yang, Qingyao Ai, Hamed Zamani, Chen Wu, W Bruce Croft, and Xueqi Cheng. 2020. A deep look into neural ranking models for information retrieval. *Information Processing & Management* 57, 6 (2020), 102067.
[7] Sebastian Hofstätter, Sophia Althammer, Michael Schröder, Mete Sertkan, and Allan Hanbury. 2020. Improving efficient neural ranking models with cross-architecture knowledge distillation. *arXiv preprint arXiv:2010.02666* (2020).
[8] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. 2021. Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling. *arXiv preprint arXiv:2104.06967* (2021).
[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* (2019).
[10] Hang Li, Ahmed Mourad, Shengyao Zhuang, Bevan Koopman, and Guido Zuccon. 2021. Pseudo relevance feedback with deep language models and dense retrievers: Successes and pitfalls. *arXiv preprint arXiv:2108.11044* (2021).
[11] Hang Li, Shengyao Zhuang, Ahmed Mourad, Xueguang Ma, Jimmy Lin, and Guido Zuccon. 2022. Improving Query Representations for Dense Retrieval with Pseudo Relevance Feedback: A Reproducibility Study. In *Proceedings of the 44rd European Conference on Information Retrieval*.
[12] Sheng-Chieh Lin, Jheng-Hong Yang, and Jimmy Lin. 2020. Distilling dense representations for ranking using tightly-coupled teachers. *arXiv preprint arXiv:2010.11386* (2020).
[13] Sheng-Chieh Lin, Jheng-Hong Yang, and Jimmy Lin. 2021. In-batch negatives for knowledge distillation with tightly-coupled teachers for dense retrieval. In *Proceedings of the 6th Workshop on Representation Learning for NLP (RepL4NLP-2021)*. 163–173.
[14] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *Workshop on Cognitive Computing at NIPS*.
[15] Ramith Padaki, Zhuyun Dai, and Jamie Callan. 2020. Rethinking Query Expansion for BERT Reranking. In *Advances in Information Retrieval-42nd European Conference on IR Research, ECIR 2020*.
[16] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2020. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. *arXiv preprint arXiv:2010.08191* (2020).
[17] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models Are Unsupervised Multitask Learners. In *OpenAI blog*.
[18] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21 (2020), 1–67.
[19] Nils Reimers, Iryna Gurevych, Nils Reimers, Iryna Gurevych, Nandan Thakur, Nils Reimers, Johannes Daxenberger, Iryna Gurevych, Nils Reimers, Iryna Gurevych, et al. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
[20] Ruiyang Ren, Yingqi Qu, Jing Liu, Wayne Xin Zhao, Qiaoqiao She, Hua Wu, Haifeng Wang, and Ji-Rong Wen. 2021. RocketQAv2: A Joint Training Method for Dense Passage Retrieval and Passage Re-ranking. *arXiv preprint arXiv:2110.07367* (2021).
[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
[22] Junmei Wang, Min Pan, Tingting He, Xiang Huang, Xueyan Wang, and Xinhui Tu. 2020. A Pseudo-Relevance Feedback Framework Combining Relevance Matching and Semantic Matching for Information Retrieval. *Information Processing &*

Management 57, 6 (2020), 102342.

[23] Xiao Wang, Craig Macdonald, Nicola Tonellotto, and Iadh Ounis. 2021. Pseudo-Relevance Feedback for Multiple Representation Dense Retrieval. *arXiv preprint arXiv:2106.11251* (2021).

[24] Thomas Wolf, Julien Chaumond, Lysandre Debut, Victor Sanh, Clement Delangue, Anthony Moi, Pierric Cistac, Morgan Funtowicz, Joe Davison, Sam Shleifer, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 38–45.

[25] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate Nearest Neighbor Negative Contrastive Learning For Dense Text Retrieval. *arXiv preprint arXiv:2007.00808* (2020).

[26] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *Advances in Neural Information Processing Systems* 32 (2019), 5753–5763.

[27] Andrew Yates, Rodrigo Nogueira, and Jimmy Lin. 2021. Pretrained Transformers for Text Ranking: BERT and Beyond. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 1154–1156.

[28] HongChien Yu, Zhuyun Dai, and Jamie Callan. 2021. PGT: Pseudo Relevance Feedback Using a Graph-Based Transformer. In *European Conference on Information Retrieval*.

[29] HongChien Yu, Chenyan Xiong, and Jamie Callan. 2021. Improving Query Representations for Dense Retrieval with Pseudo Relevance Feedback. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management*.

[30] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Min Zhang, and Shaoping Ma. 2020. RepBERT: Contextualized Text Embeddings for First-Stage Retrieval. *arXiv preprint arXiv:2006.15498* (2020).

[31] Zhi Zheng, Kai Hui, Ben He, Xianpei Han, Le Sun, and Andrew Yates. 2020. BERT-QE: Contextualized Query Expansion for Document Re-ranking. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings*. 4718–4728.

[32] Zhi Zheng, Kai Hui, Ben He, Xianpei Han, Le Sun, and Andrew Yates. 2021. Contextualized query expansion via unsupervised chunk selection for text retrieval. *Information Processing & Management* 58, 5 (2021), 102672.